

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ
им. А.И. ГЕРЦЕНА»

Направление подготовки

09.03.01 – Информатика и вычислительная техника

Профиль «Технологии разработки программного обеспечения»

Лабораторная работа №4

«Численные методы решения дифференциальных уравнений»

Работу выполнили студенты 2 курса 1.1 группы:

Шульга Евгений

Зубов Михаил

САНКТ-
ПЕТЕРБУРГ 2024

Часть 1. Численные методы решения дифференциальных уравнений 1-го порядка.

Цель: разработать программы решения дифференциальных уравнений с использованием численных методов Эйлера и Рунге-Кутты.

Оборудование: ПК, Pycharm, Word

Постановка задачи: изучить численные методы Эйлера и Рунге-Кутты и предложенные варианты алгоритмов их реализации. Разработать программы решения дифференциальных уравнений, используя актуальный для студента язык программирования.

Ход работы:

В ходе изучения материалов лекции мы разработали алгоритм решения дифференциальных уравнений методом Эйлера (п 1.1) и Рунге-Кутты (п. 1.2), дополнили программу библиотекой Matplotlib для отрисовки интегральной кривой дифференциального уравнения.

1.1 «Метод Эйлера»

Код программы:

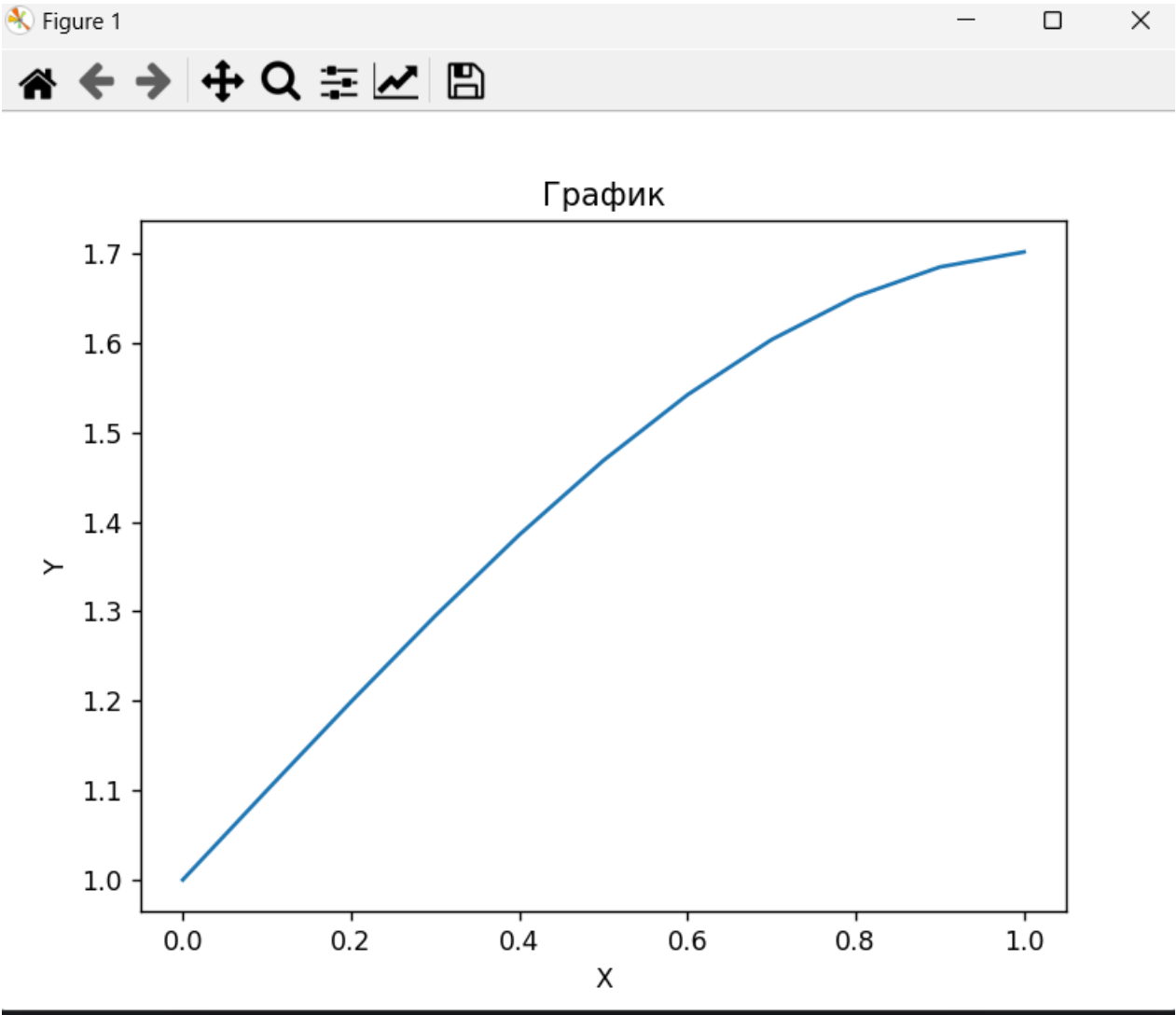
```
def Ailer_first_order(a, b, y0, equation, n):
    h = abs(a - b) / n
    x = a
    y = y0
    answersx = []
    answersy = []
    while x - x % 0.000000000000001 <= b:
        answersx.append(x)
        answersy.append(y)
        y += h * solve_even(x, y, equation)
        x += h
    return answersx, answersy
```

Начальные условия для контрольного примера:

Имя переменной	Тип	Смысл	Значение
Equation	string	Уравнение в формате строки	$y' = y*(1 - x)$
a	int	Нижняя граница интервала	0
b		Верхняя граница интервала	1
y0		Начальное значение времени	1
n		Количество	10

		разбиений	
--	--	-----------	--

Пример работы программы:



```

Введите a: 0
Введите b: 1
Введите y0: 1
Введите выражение: y*(1-x)
Введите n: 10
x: [0.0, 0.1, 0.2, 0.30000000000000004, 0.4, 0.5, 0.6, 0.7, 0.7999999999999999, 0.8999999999999999, 0.9999999999999999]
y: [1.0, 1.1, 1.199, 1.29492, 1.3855644, 1.468698264, 1.5421331772, 1.603818504288, 1.65193305941664, 1.6849717206049728, 1.7018214378110226]

```

1.2«Метод Рунге-Кутта»

Код программы:

```

def Runge_first_order(a, b, y0, equation, n):
    h = abs(a - b) / n
    x = a
    y = y0
    answerx = []
    answey = []
    while x - x % 0.000000000000001 < b:

```

```

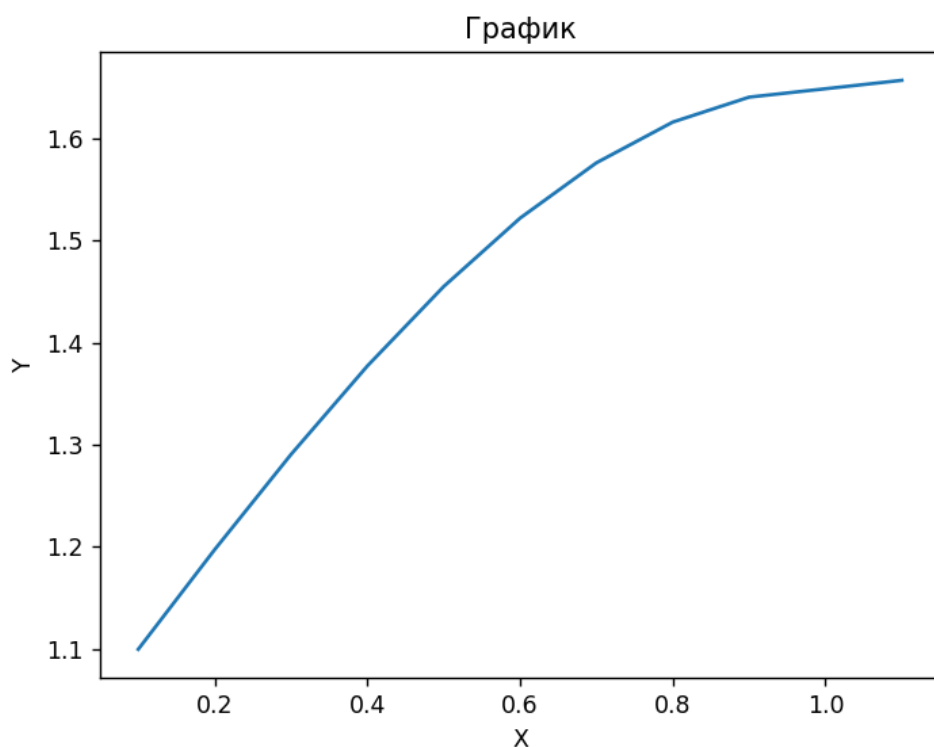
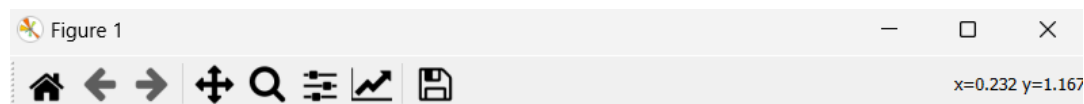
k1 = h * solve_even(x, y, equation)
k2 = h * solve_even(x + h / 2, y + k1 / 2, equation)
k3 = h * solve_even(x + h / 2, y + k2 / 2, equation)
k4 = h * solve_even(x + h, y + k3, equation)
F = (k1 + 2 * k2 + 2 * k3 + k4) / 6
x += h
y += F
answerx.append(x)
answery.append(y)
return answerx, answery

```

Начальные условия для контрольного примера:

Имя переменной	Тип	Смысл	Значение
Equation	string	Уравнение в формате строки	$y' = y*(1 - x)$
a	int	Нижняя граница интервала	0
b		Верхняя граница интервала	1
y0		Начальное значение времени	1
n		Количество разбиений	10

Пример работы программы:



```

Введите a: 0
Введите b: 1
Введите y0: 1
Введите выражение: y*(1-x)
Введите n: 10
x: [0.1, 0.2, 0.30000000000000004, 0.4, 0.5, 0.6, 0.7, 0.7999999999999999, 0.8999999999999999, 0.9999999999999999, 1.0999999999999999]
y: [1.0996587802083333, 1.1972172320380163, 1.2904614485902193, 1.3771275621895505, 1.45499119087384, 1.5219613162523975, 1.5761731325294723, 1.6160741441918625, 1.64049797676

```

1.3 «Оценка проделанной работы»

Сравнив работу двух функций программы, мы можем привести качественную оценку обоих методов:

Как и следовало ожидать, метод рунге-Кутта дает наиболее точный результат, при увеличении количества разбиений, можно добиться более точного значения Эйлера.

Часть 2. Численные методы решения дифференциальных уравнений 2-го порядка. Системы Дифференциальных Уравнений.

Цель: разработать программы решения дифференциальных уравнений второго порядка и системы дифференциальных уравнений с использованием численных методов Эйлера и Рунге-Кутта.

Оборудование: ПК, язык программирования (выбирает студент).

Постановка задачи: изучить алгоритмы решения дифференциальных уравнений высших порядков (второго порядка) и системы дифференциальных уравнений с использованием численных методов Эйлера и Рунге-Кутта. Разработать программы, используя актуальный для студента язык программирования

Ход работы:

В ходе изучения материалов лекции мы разработали алгоритм решения дифференциальных уравнений высшего порядка методом Эйлера (п 2.1) и Рунге-Кутта (п. 2.2), Системы ДУ (п. 2.3) дополнили программу библиотекой Matplotlib для отрисовки интегральной кривой дифференциального уравнения.

2.1«Метод Эйлера»

Код программы:

```

def Ailer_second_order(a, b, y0, dy, equation, n):
    h = abs(a - b) / n
    x = a
    y = y0
    z = dy
    answerx = []

```

```

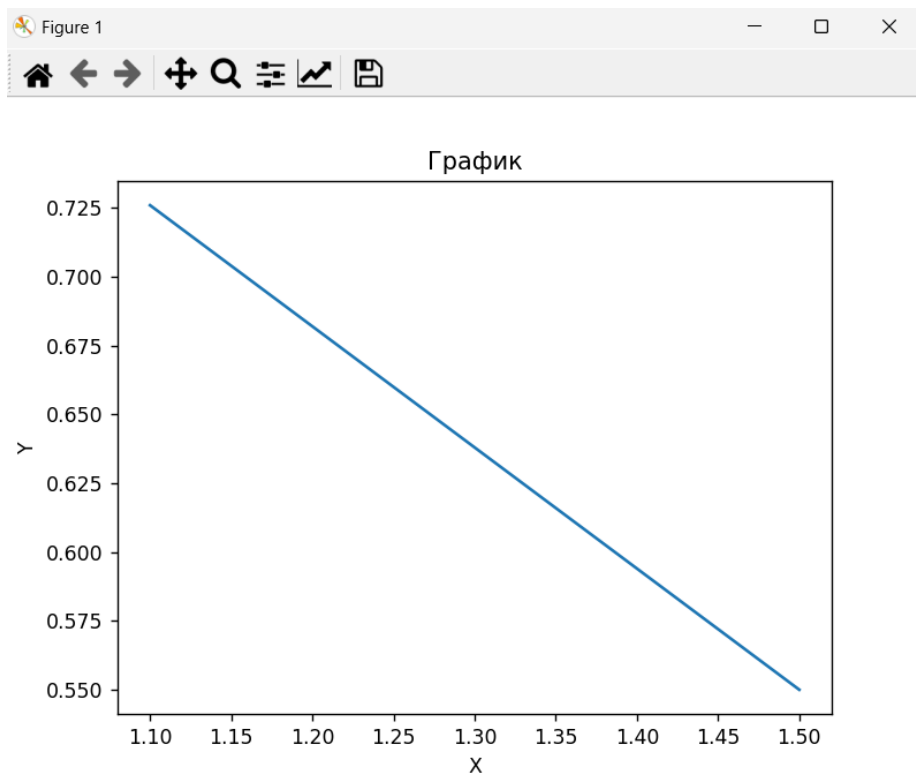
answery = []
while x - x % 0.000000000000001 < b:
    y += h * z
    z += h * second_solve(x, y, z, equation) * -1
    x += h
    answerx.append(x)
    answery.append(y)
return answerx, answery

```

Начальные условия для контрольного примера:

Имя переменной	Тип	Смысл	Значение
Equation	string	Уравнение в формате строки	-dy/x-y
a	float	Нижняя граница интервала	1
b	float	Верхняя граница интервала	1.5
y0	Float	Начальное значение времени	0.77
dy0	float	Начальное значение времени	-0.44
n	int	Количество разбиений	5

Пример работы программы:

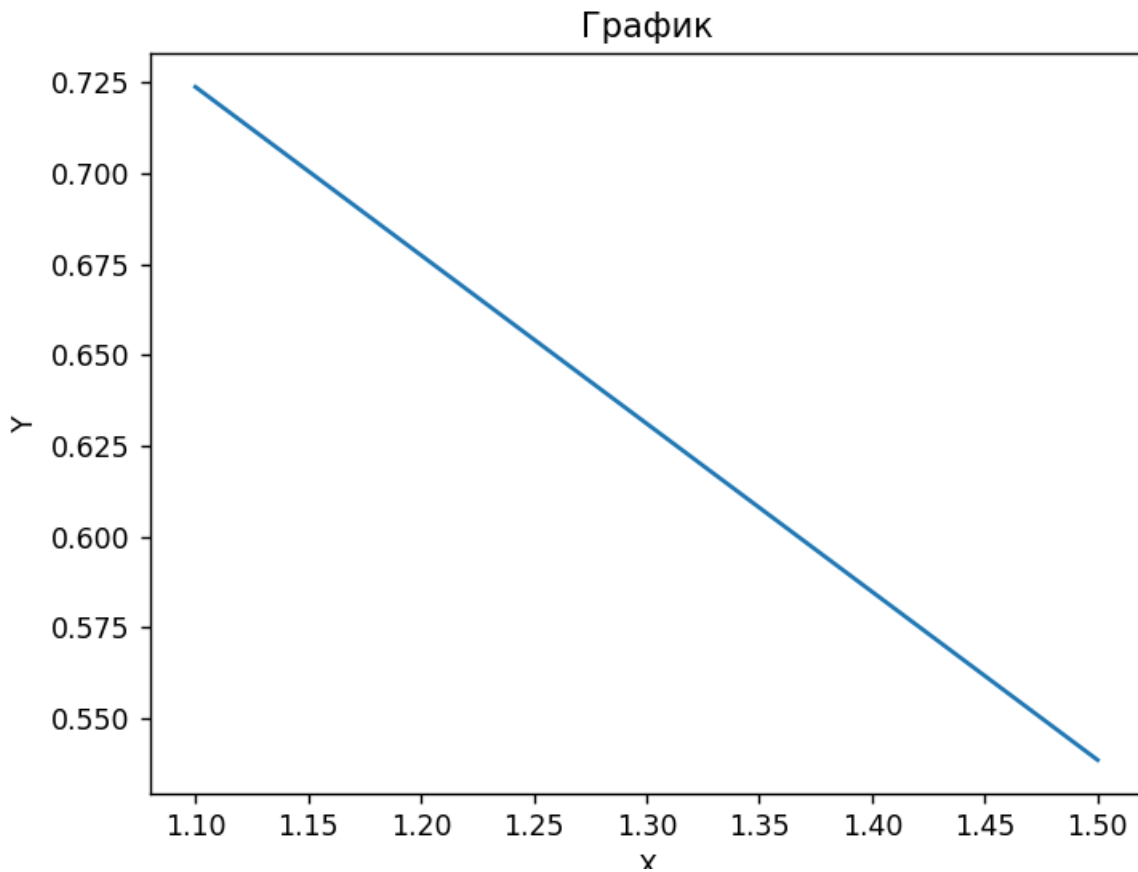
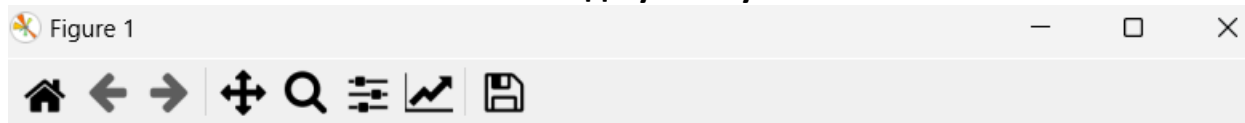


```

Введите a: 1
Введите b: 1.5
Введите y0: 0.77
Введите dy: -0.44
Введите выражение: -z/x-y
Введите n: 5
x: [1.1, 1.2000000000000002, 1.3000000000000003, 1.4000000000000004, 1.5000000000000004]
y: [0.726, 0.6819999999999999, 0.6379999999999999, 0.5939999999999999, 0.5499999999999998]

```

2.2«Метод Рунге-Кутта»



Код программы:

```

def Runge_second_order(x0, b, y0, dy0, equation, n):
    h = abs(x0 - b) / n
    x = x0
    y = y0
    z = dy0
    answerx = []
    answey = []
    while x - x % 0.0000000000000001 < b:
        k1 = h * z
        k2 = h * (z + k1 / 2)
        k3 = h * (z + k2 / 2)
        k4 = h * (z + k3)
        F = (k1 + 2 * k2 + 2 * k3 + k4) / 6

```

```

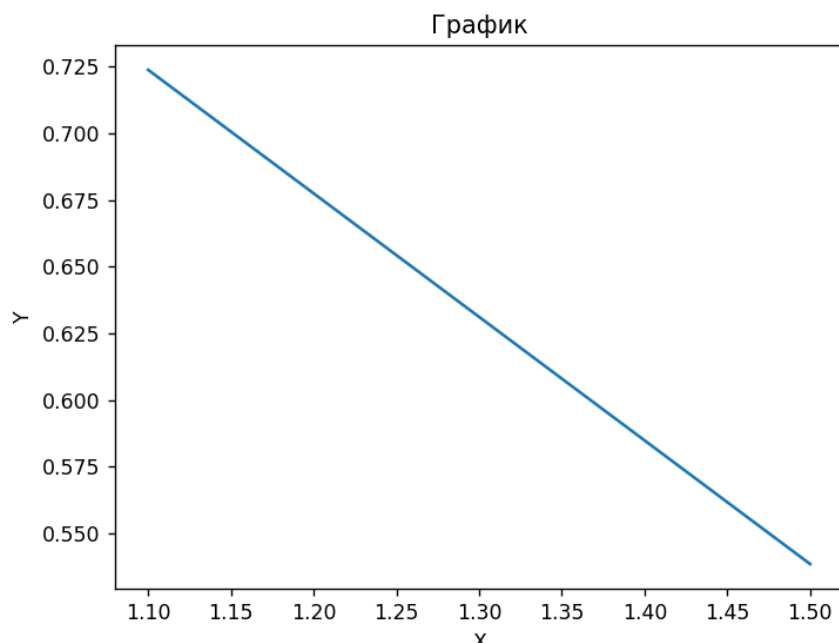
y += F
k1 = h * second_solve(x, y, z, equation) * -1
k2 = h * second_solve(x + h / 2, y + k1 / 2, z + k1 / 2, equation) * -1
k3 = h * second_solve(x + h / 2, y + k2 / 2, z + k2 / 2, equation) * -1
k4 = h * second_solve(x + h, y + k3, z + k3, equation) * -1
F = (k1 + 2 * k2 + 2 * k3 + k4) / 6
z += F
x += h
answerx.append(x)
answery.append(y)
return answerx, answery

```

Начальные условия для контрольного примера:

Имя переменной	Тип	Смысл	Значение
Equation	string	Уравнение в формате строки	-dy/x-y
a	float	Нижняя граница интервала	1
b	float	Верхняя граница интервала	1.5
y0	Float	Начальное значение времени	0.77
dy0	float	Начальное значение времени	-0.44
n	int	Количество разбиений	5

Пример работы программы:



```

Введите a: 1
Введите b: 1.5
Введите y0: 0.77
Введите dy: -0.44
Введите выражение: -z/x-y
Введите n: 5
x: [1.1, 1.2000000000000002, 1.3000000000000003, 1.4000000000000004, 1.5000000000000004]
y: [0.7237248333333334, 0.6774496666666667, 0.6311745000000001, 0.5848993333333334, 0.5386241666666668]

```

Проанализировав работу двух функций программы, мы приходим к тому же выводу, что и при первой части.

2.3«Система ДУ»

Код программы:

```

def defur_sistem(x0, b, fy1, fy2, fy3, eq1, eq2, eq3, n):
    h = abs(x0 - b) / n
    t = x0
    y1 = fy1
    y2 = fy2
    y3 = fy3
    answert = []
    answerx = []
    answey = []
    answerz = []
    oldy1 = 0
    oldy2 = 0
    oldy3 = 0
    while t - t % 0.0000000000000001 <= b:
        answert.append(t)
        answerx.append(y1)
        answey.append(y2)
        answerz.append(y3)
        oldy1 = y1
        oldy2 = y2
        oldy3 = y3
        y1 += h * sissolve(t, oldy1, oldy2, oldy3, eq1)
        y2 += h * sissolve(t, oldy1, oldy2, oldy3, eq2)
        y3 += h * sissolve(t, oldy1, oldy2, oldy3, eq3)
        t += h
    return answert, answerx, answey, answerz

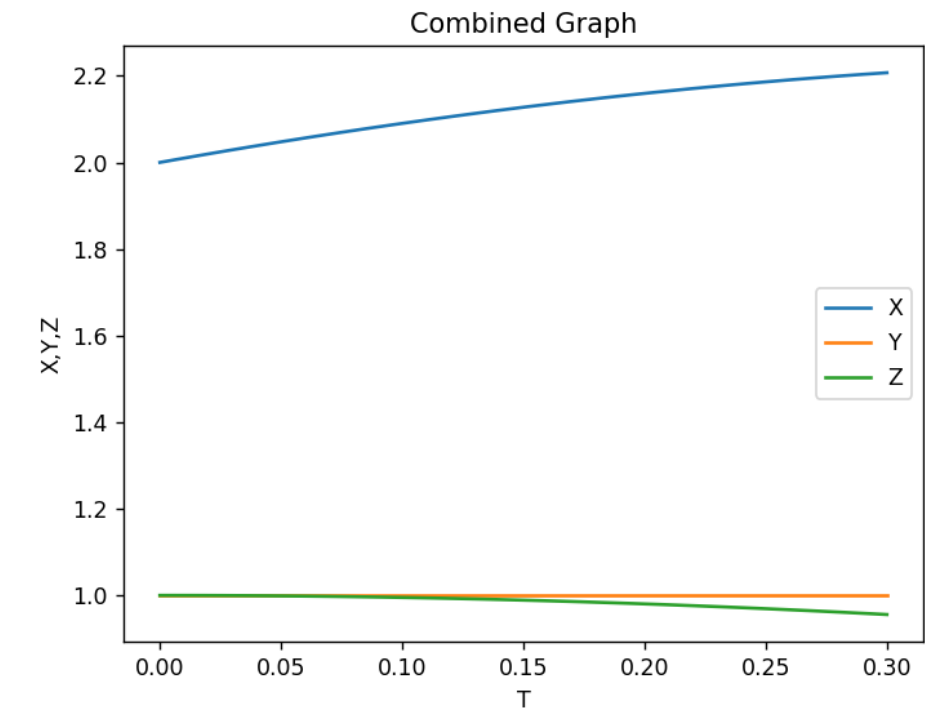
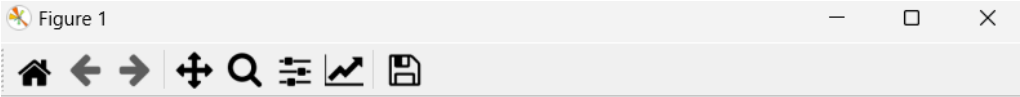
```

Начальные условия для контрольного примера:

Имя переменной	Тип	Смысл	Значение
Eq1	string	Уравнение в формате строки	-2*x+5*z
Eq2	int	Уравнение в формате строки	Sin(t-1)*x-y+3*z
Eq3		Уравнение в формате строки	-x+2*z
Fy1		Начальные значения	2
Fy2			1

Fy3			1
X0		Нижняя граница	0
b		Верхняя граница	0,3
n		Количество разбиений	100

Пример работы программы:



```
Введите x0: 0
Введите b: 0.3
Введите y1: 2
Введите y2: 1
Введите y3: 1
Введите 1-ое выражение: -2*x+5*z
Введите 2-ое выражение: sin(t-1)*x-y+3*z
Введите 3-ое выражение: -x+2*z
Введите n: 100
t: [0.0, 0.003, 0.006, 0.009000000000000001, 0.012, 0.015, 0.018, 0.020999999999999998, 0.023999999999999997, 0.026999999999999996, 0.029999999999999995, 0.032999999999999994,
x: [2.0, 2.003, 2.005982, 2.008945973, 2.0118918921619997, 2.0148197308102427, 2.0177294624314563, 2.0206210606750927, 2.0234944993535673, 2.0263497524424956, 2.02918679408092,
y: [2.0, 2.003, 2.005982, 2.008945973, 2.0118918921619997, 2.0148197308102427, 2.0177294624314563, 2.0206210606750927, 2.0234944993535673, 2.0263497524424956, 2.02918679408092,
z: [2.0, 2.003, 2.005982, 2.008945973, 2.0118918921619997, 2.0148197308102427, 2.0177294624314563, 2.0206210606750927, 2.0234944993535673, 2.0263497524424956, 2.02918679408092]
```

Вывод: Мы научились решать дифференциальные уравнения 1-го и 2-го порядка

методами Эйлера и Рунге-Кутты, а также системы дифференциальных уравнений с 3-мя неизвестными.